

SafeRoute Project Documentation

Introduction, Solution Overview, and Technology Stack

SafeRoute is a cross-platform transport safety system focused on women commuters in Dar es Salaam. The project combines a Flutter mobile app, a Django/PostgreSQL backend, map and location services, and cloud deployment support for local development, physical devices, and Render-hosted production.

Project	SafeRoute	Frontend	Flutter
Primary Goal	Safer route planning and route recording	Backend	Django + PostgreSQL

1. Introduction

- SafeRoute addresses the real-world problem of transport poverty and safety uncertainty faced by women commuters. The application is designed to help users identify safer travel options, record journeys, and preserve a history of route behavior that can later support safety analysis and operational planning.
- The current implementation is not a simple static map app. It is a complete system with user authentication, session handling, route recording, location naming, GPS tracking, database-backed persistence, and an admin dashboard for reviewing recorded travel and managing users.

Current step in the project: the app is being stabilized around local PostgreSQL and Render deployment behavior, with the backend configured to use environment-driven database settings and the UI starting at the login flow before any dashboard access.

2. Problem Statement

- Safety visibility: commuters often lack a clear view of which routes, stops, or corridors are safer at specific times of day.
- Weak historical data: without structured route records, it is difficult to analyze travel behavior, repeated incidents, or risky locations.
- Deployment friction: the app must work across local development, Android devices, emulators, and Render-hosted backend environments.

3. Elaborate Solution

SafeRoute solves the problem with a layered architecture. The Flutter client provides an accessible mobile experience for route discovery, recording, and feedback. The Django backend exposes JSON APIs for login, registration, route recording, and route retrieval. PostgreSQL stores users, routes, travel logs, safety reports, and incident records in a durable relational model.

4. Architecture Overview

- Frontend: Flutter app with Material 3 theming, localization, Google Maps integration, location tracking, and persistent session storage.
- Backend: Django 5.x API using JSON endpoints, CSRF-exempt mobile routes, Google sign-in verification, and a simple admin dashboard.
- Database: PostgreSQL with a dedicated saferoute schema and tables for users, routes, logs, locations, and safety reports.
- Deployment: local development on Windows, optional Android emulators/devices, and Render deployment for production hosting.

5. Technology Stack

- Flutter
- Dart 3.10+
- Django 5.1+
- PostgreSQL 12+
- psycopg 3
- dj-database-url
- google_sign_in
- geolocator
- google_maps_flutter
- http
- shared_preferences
- gunicorn
- Render

6. What Has Been Implemented So Far

- Flutter app bootstrapping with splash, auth, and home flow.
- Session persistence in shared_preferences.
- Backend API client with fallback URLs for local and remote deployment.
- Django authentication endpoints for email/password and Google ID token login.
- Route recording API that persists named locations and coordinates.
- Render deployment configuration and environment variable support.
- Admin login root route and trailing-slash-compatible URL patterns.
- Database connection troubleshooting and local PostgreSQL verification.

7. Key Design Notes

- The app favors environment-driven configuration rather than hard-coded backend hosts.
- The backend is structured to support both local development and public deployment.
- The project uses a relational database because route history and user relationships are naturally tabular.
- The interface is designed for practical mobile use, not only desktop admin access.

8. Current Status and Next Steps

- At this step, the project is functionally structured but still being refined around deployment and runtime correctness.
- The immediate focus is making sure the backend points at the correct PostgreSQL instance, that the API responds correctly on startup, and that the documentation reflects the real architecture rather than a placeholder summary.

Recommended next steps: verify the deployed database credentials, confirm the backend health endpoint, and then expand the documentation with screenshots and implementation examples.